

Supplementary Material for “Interpretable Clustering-based Unsupervised Software Defect Prediction”

1 Software Projects and Metrics

In this paper, we conduct defect prediction on 28 releases across 8 open-source software projects from the Apache community [1]. Each project consists of 65 software metrics along 3 dimensions, i.e., 54 code metrics, 5 process metrics, and 6 ownership metrics. Among them, code metrics (e.g., LOC and cyclomatic complexity) describe the relationship between code properties and software quality, process metrics (e.g., the number of added lines and deleted lines) describe the relationship between development process and software quality, and ownership metrics (e.g., the number of own commits and own lines) describe the relationship between the ownership of modules and software quality. Before model training, AutoSpearman is applied to remove redundant metrics, and the resulting feature set is used in subsequent experiments. Tables 1 and 2 separately show the statistical summary of the studied software projects and metrics.

Table 1 An overview of the used software projects

Project	#Metrics	#Files	#Defective files	%Defective files
activemq-5.0.0	65	1884	293	15.55%
activemq-5.1.0	65	1970	154	7.82%
activemq-5.2.0	65	2040	219	10.74%
activemq-5.3.0	65	2367	258	10.90%
activemq-5.8.0	65	3420	206	6.02%
derby-10.2.1.6	65	1963	661	33.67%
derby-10.3.1.4	65	2206	669	30.33%
derby-10.5.1.1	65	2705	383	14.16%
groovy-1_5_7	65	757	26	3.43%
groovy-1_6_BETA_1	65	821	70	8.53%
groovy-1_6_BETA_2	65	884	76	8.60%
hbase-0.94.0	65	1059	218	20.59%
hbase-0.95.0	65	1669	383	22.95%
hbase-0.95.2	65	1834	483	26.34%
hive-0.10.0	65	1560	176	11.28%
hive-0.12.0	65	2662	213	8.00%
hive-0.9.0	65	1416	283	19.99%
jruby-1.1	65	731	87	11.90%
jruby-1.4.0	65	978	180	18.40%
jruby-1.5.0	65	1131	82	7.25%
jruby-1.7.0.	65	1614	87	5.39%
lucene-2.3.0	65	805	196	24.35%
lucene-2.9.0	65	1368	273	19.96%
lucene-3.0.0	65	1337	155	11.59%
lucene-3.1	65	2806	107	3.81%
wicket-1.3.0-beta2	65	1763	130	7.37%
wicket-1.3.0-incubating-beta-1	65	1672	101	6.04%
wicket-1.5.3	65	2578	105	4.07%

Table 2 An overview of the used software metrics

Metric type	Metric name	Description
Code metrics (54)	CountDeclMethodPrivate	the number of local (not inherited) private methods
	AvgLineCode	the average number of lines containing source code for all nested functions or methods
	CountLine	the number of physical lines
	MaxCyclomatic	the maximum cyclomatic complexity of all nested functions or methods
	CountDeclMethodDefault	the number of local default methods
	AvgEssential	the average essential complexity for all nested functions or methods
	CountDeclClassVariable	the number of class variables
	SumCyclomaticStrict	the sum of strict cyclomatic complexity of all nested functions or methods
	AvgCyclomatic	the average cyclomatic complexity for all nested functions or methods
	AvgLine	the average number of lines for all nested functions or methods
	CountDeclClassMethod	the number of class methods
	AvgLineComment	the average number of lines containing comment for all nested functions or methods
	AvgCyclomaticModified	the modified McCabe Cyclomatic complexity
	CountDeclFunction	the number of functions
	CountLineComment	the number of lines containing comment
	CountDeclClass	the number of classes
	CountDeclMethod	the number of local (not inherited) methods
	SumCyclomaticModified	the sum of modified cyclomatic complexity of all nested functions or methods
	CountLineCodeDecl	the number of lines containing declarative source code
	CountDeclMethodProtected	the number of local protected methods
	CountDeclInstanceVariable	the number of instance variables
	MaxCyclomaticStrict	the maximum strict cyclomatic complexity of nested functions or methods
	CountDeclMethodPublic	the number of local (not inherited) public methods
	CountLineCodeExe	the number of lines containing executable source code
	SumCyclomatic	the sum of cyclomatic complexity of all nested functions or methods
	SumEssential	the sum of essential complexity of all nested functions or methods
	CountStmtDecl	the number of declarative statements
	CountLineCode	the number of lines containing source code
	CountStmtExe	the number of executable statements
	RatioCommentToCode	the ratio of comment lines to code lines
	CountLineBlank	the number of blank lines
	CountStmt	the number of statements
	MaxCyclomaticModified	the maximum modified cyclomatic complexity of nested functions or methods
	CountSemicolon	the number of semicolons
	AvgLineBlank	the average number of blank lines for all nested functions or methods
	CountDeclInstanceMethod	the number of instance methods
	AvgCyclomaticStrict	the average strict cyclomatic complexity for all nested functions or methods
	PercentLackOfCohesion	the 100% minus the average cohesion for package entities
	MaxInheritanceTree	the maximum depth of class in inheritance tree
	CountClassDerived	the number of immediate subclasses
	CountClassCoupled	the number of other classes to which a class is coupled
	CountClassBase	the number of immediate base classes
	CountInput (max, min, mean)	the number of calling subprograms plus global variables read
	CountOutput (max, min, mean)	the number of called subprograms plus global variables set
	CountPath (max, min, mean)	the number of unique paths through a body of code, not counting abnormal exits or gotos
	MaxNesting (max, min, mean)	the maximum nesting level of control constructs
Process metrics (5)	COMM	the number of Git commits
	ADEV	the number of active developers
	DDEV	the number of distinct developers
	Added_lines	the normalized number of lines added to the module
	Del_lines	the number normalized of lines deleted to the module
Ownership metrics (6)	OWN_LINE	the proportion of lines of code written by the developer who has the highest contribution of lines of code on the module
	OWN_COMMIT	the proportion of code changes (i.e., Git commits) made by the developer who has the highest contribution of code changes on the module
	MINOR_COMMIT	the number of unique developers who have contributed less than 5% of the total code changes (i.e., Git commits) on the module
	MINOR_LINE	the number of unique developers who have contributed less than 5% of the total lines of code on the module
	MAJOR_COMMIT	the number of unique developers who have contributed more than 5% of the total code changes (i.e., Git commits) on the module
	MAJOR_LINE	the number of unique developers who have contributed more than 5% of the total lines of code on the module

2 Metrics After Correlation and Redundancy Analysis

Prior work [2] points out that correlated or redundant software metrics impact the performance and interpretation of defect prediction models. Therefore, we conduct correlation and redundancy analyses prior to building our ICUSDP models. Similar to the previous defect prediction studies [3] [4] [5], we use the *AutoSpearman* algorithm [6] to remove the correlated and redundant metrics during data preprocessing. The remaining software metrics after applying the *AutoSpearman* algorithm are shown in Table 3. From the table, we can observe that more than half of the software metrics have been removed for each project. Subsequently, we use these metrics to perform unsupervised defect prediction experiments with ICUSDP.

Table 3 An overview of the selected metrics used by AutoSpearman

Dataset	Metrics
activemq-5.0.0	AvgCyclomaticModified, AvgEssential, AvgLineBlank, AvgLineComment, COMM, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountOutput_Min, MAJOR_LINE, MINOR_COMMIT, MaxInheritanceTree, MaxNesting_Min, OWN_COMMIT, OWN_LINE, PercentLackOfCohesion, RatioCommentToCode
activemq-5.1.0	AvgCyclomaticModified, AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountOutput_Min, MAJOR_LINE, MINOR_COMMIT, MaxInheritanceTree, MaxNesting_Min, OWN_COMMIT, OWN_LINE, PercentLackOfCohesion, RatioCommentToCode
activemq-5.2.0	AvgCyclomaticModified, AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Min, Del_lines, MAJOR_LINE, MINOR_COMMIT, MaxInheritanceTree, MaxNesting_Min, OWN_LINE, PercentLackOfCohesion, RatioCommentToCode
activemq-5.3.0	AvgCyclomaticModified, AvgEssential, AvgLineBlank, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Mean, CountOutput_Min, MAJOR_LINE, MINOR_COMMIT, MaxInheritanceTree, MaxNesting_Min, OWN_COMMIT, OWN_LINE, PercentLackOfCohesion, RatioCommentToCode
activemq-5.8.0	AvgCyclomaticModified, AvgEssential, AvgLineBlank, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Mean, CountOutput_Min, Del_lines, MAJOR_LINE, MINOR_COMMIT, MaxInheritanceTree, MaxNesting_Min, OWN_COMMIT, OWN_LINE, PercentLackOfCohesion, RatioCommentToCode
derby-10.2.1.6	Added_lines, AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Min, CountPath_Min, MAJOR_LINE, MINOR_COMMIT, MaxInheritanceTree, MaxNesting_Mean, OWN_LINE, PercentLackOfCohesion, RatioCommentToCode
derby-10.3.1.4	Added_lines, AvgEssential, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Min, CountPath_Min, MAJOR_LINE, MINOR_COMMIT, MaxInheritanceTree, MaxNesting_Mean, OWN_COMMIT, OWN_LINE, PercentLackOfCohesion, RatioCommentToCode
derby-10.5.1.1	AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Min, Del_lines, MAJOR_LINE, MINOR_COMMIT, MINOR_LINE, MaxInheritanceTree, MaxNesting_Mean, MaxNesting_Min, OWN_COMMIT, PercentLackOfCohesion, RatioCommentToCode
groovy-1_5_7	AvgCyclomaticStrict, AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass,

[illegible]

1.7.0.preview1	CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Mean, CountOutput_Min, CountPath_Min, Del_lines, MAJOR_LINE, MINOR_COMMIT, MINOR_LINE, MaxInheritanceTree, MaxNesting_Mean, MaxNesting_Min, OWN_COMMIT, PercentLackOfCohesion, RatioCommentToCode
lucene-2.3.0	AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountOutput_Min, CountPath_Min, MAJOR_LINE, MINOR_COMMIT, MINOR_LINE, MaxInheritanceTree, MaxNesting_Mean, OWN_COMMIT, PercentLackOfCohesion, RatioCommentToCode
lucene-2.9.0	Added_lines, AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountOutput_Mean, CountOutput_Min, CountPath_Min, MAJOR_LINE, MINOR_COMMIT, MINOR_LINE, MaxInheritanceTree, MaxNesting_Mean, OWN_COMMIT, PercentLackOfCohesion, RatioCommentToCode
lucene-3.0.0	Added_lines, AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Mean, CountOutput_Min, MAJOR_COMMIT, MAJOR_LINE, MINOR_COMMIT, MINOR_LINE, MaxInheritanceTree, MaxNesting_Mean, MaxNesting_Min, OWN_COMMIT, PercentLackOfCohesion, RatioCommentToCode
lucene-3.1	Added_lines, AvgEssential, AvgLineBlank, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclClassVariable, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Mean, CountOutput_Min, MAJOR_LINE, MINOR_COMMIT, MINOR_LINE, MaxInheritanceTree, MaxNesting_Mean, MaxNesting_Min, OWN_COMMIT, PercentLackOfCohesion, RatioCommentToCode
wicket-1.3.0-beta2	AvgCyclomaticModified, AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Min, Del_lines, MAJOR_LINE, MINOR_COMMIT, MINOR_LINE, MaxInheritanceTree, MaxNesting_Min, OWN_COMMIT, PercentLackOfCohesion, RatioCommentToCode
wicket-1.3.0-incubating-beta-1	AvgCyclomaticModified, AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountInput_Mean, CountInput_Min, CountLineComment, CountOutput_Min, Del_lines, MAJOR_LINE, MINOR_COMMIT, MINOR_LINE, MaxInheritanceTree, MaxNesting_Min, OWN_COMMIT, PercentLackOfCohesion, RatioCommentToCode
wicket-1.5.3	AvgEssential, AvgLineBlank, AvgLineComment, CountClassBase, CountClassCoupled, CountClassDerived, CountDeclClass, CountDeclClassMethod, CountDeclInstanceVariable, CountDeclMethodDefault, CountDeclMethodPrivate, CountDeclMethodProtected, CountDeclMethodPublic, CountInput_Mean, CountInput_Min, CountOutput_Min, CountPath_Min, MAJOR_LINE, MINOR_COMMIT, MaxInheritanceTree, MaxNesting_Mean, MaxNesting_Min, OWN_COMMIT, OWN_LINE, PercentLackOfCohesion, RatioCommentToCode

3 Results

3.1 Detailed Results for RQ1

Tables 4 and 5 present the median results of the ICUSDP and unsupervised methods in terms of MCC and F1@20%LOC across all 28 datasets. The NPSKESD test is applied to determine statistical significance and to identify groups of methods that are not significantly different.

In the unsupervised setting, ICUSDP achieves competitive performance on both measures. For MCC, ICUSDP obtains a median value of 0.268, which is close to CLA (0.285), SC (0.270), and TCL (0.269). This narrow margin indicates that its overall discriminative ability is comparable to that of the strongest unsupervised competitors. For F1@20%LOC, ICUSDP obtains a median value of 0.212, lower than TCL (0.263) and CLAMI (0.235), but higher than most other unsupervised methods. This suggests that its effort-aware ranking is reasonably effective, although there remains room for improvement relative to the top-performing baselines. Moreover, ICUSDP is included in the statistically best group on

several datasets, indicating that its competitiveness is not limited to average trends but can also hold on individual projects.

Overall, these results indicate that ICUSDP provides competitive performance among unsupervised methods and remains applicable to defect prediction scenarios where historical defect labels are unavailable or limited. This is practically important because it suggests that useful defect prediction can be obtained without relying on labeled historical data, while the remaining gap to the strongest baselines highlights opportunities for further improving effort-aware ranking.

Table 4 Comparison results of unsupervised methods using the NPSKESD test in terms of MCC.

Dataset	ONE	CLA	CLAMI	MD	MU	SC	TCL	TCLP	KMedoids	ICUSDP
activemq-5.0.0	0.291	0.338	0.243	0.265	-0.271	0.360	0.352	0.296	0.184	0.365
activemq-5.1.0	0.203	0.264	0.184	0.228	-0.232	0.242	0.296	0.277	0.116	0.247
activemq-5.2.0	0.254	0.244	0.165	0.226	-0.226	0.240	0.255	0.217	0.108	0.232
activemq-5.3.0	0.207	0.288	0.223	0.225	-0.229	0.293	0.301	0.223	0.127	0.295
activemq-5.8.0	0.172	0.218	0.196	0.196	-0.202	0.235	0.235	0.232	0.037	0.242
derby-10.2.1.6	0.339	0.402	0.394	0.347	-0.348	0.339	0.334	0.165	0.203	0.360
derby-10.3.1.4	0.294	0.377	0.330	0.316	-0.317	0.339	0.301	0.229	0.190	0.338
derby-10.5.1.1	0.295	0.346	0.298	0.263	-0.265	0.296	0.343	0.249	0.134	0.344
groovy-1_5_7	0.167	0.267	0.222	0.118	-0.121	0.196	0.339	0.378	0.101	0.206
groovy-1_6beta1	0.174	0.255	0.225	0.181	-0.186	0.236	0.242	0.229	0.134	0.225
groovy-1_6beta2	0.174	0.214	0.222	0.169	-0.175	0.196	0.233	0.242	0.158	0.199
hbase-0.94.0	0.279	0.296	0.274	0.311	-0.312	0.339	0.199	0.176	0.095	0.325
hbase-0.95.0	0.235	0.207	0.212	0.212	-0.216	0.283	0.217	0.193	0.162	0.230
hbase-0.95.2	0.269	0.271	0.278	0.278	-0.280	0.347	0.246	0.197	0.167	0.303
hive-0.10.0	0.364	0.320	0.248	0.279	-0.279	0.325	0.299	0.289	0.126	0.303
hive-0.12.0	0.150	0.173	0.110	0.142	-0.145	0.195	0.090	0.095	0.214	0.229
hive-0.9.0	0.463	0.320	0.281	0.313	-0.314	0.268	0.253	0.219	0.066	0.202
jruby-1.1	0.444	0.375	0.405	0.302	-0.302	0.275	0.492	0.350	0.205	0.379
jruby-1.4.0	0.431	0.421	0.420	0.350	-0.357	0.388	0.391	0.232	0.257	0.415
jruby-1.5.0	0.312	0.350	0.353	0.252	-0.255	0.261	0.387	0.323	0.208	0.310
jruby-1.7.0	0.256	0.244	0.306	0.212	-0.212	0.209	0.305	0.284	0.134	0.273
lucene-2.3.0	0.210	0.296	0.218	0.279	-0.281	0.302	0.191	0.182	0.210	0.276
lucene-2.9.0	0.235	0.282	0.166	0.270	-0.273	0.277	0.237	0.215	0.170	0.260
lucene-3.0.0	0.177	0.297	0.196	0.236	-0.236	0.280	0.111	0.109	0.121	0.293
lucene-3.1	0.103	0.145	0.099	0.130	-0.132	0.141	0.185	0.182	0.072	0.150
wicket-1.3.0beta2	0.236	0.304	0.194	0.230	-0.234	0.275	0.269	0.251	0.086	0.265
wicket-1.3.0beta1	0.206	0.263	0.204	0.212	-0.216	0.246	0.307	0.286	0.062	0.241
wicket-1.5.3	0.180	0.173	0.136	0.176	-0.179	0.176	0.199	0.168	0.059	0.163
Median	0.244	0.285	0.229	0.238	-0.242	0.270	0.269	0.226	0.129	0.268

(1) MD → ManualDown, and MU → ManualUp.

(2) The gray cells indicate the results that are better than others with statistical significance using the NPSKESD test in each row. If multiple treatments tie for the “best”, then there will be multiple gray cells in that row (the same below).

Table 5 Comparison results of unsupervised methods using the NPSKESD test in terms of F1@20%LOC.

Dataset	ONE	CLA	CLAMI	MD	MU	SC	TCL	TCLP	KMedoids	ICUSDP
activemq-5.0.0	0.249	0.108	0.220	0.284	0.098	0.221	0.331	0.309	0.127	0.227
activemq-5.1.0	0.162	0.047	0.175	0.158	0.041	0.130	0.277	0.227	0.077	0.145
activemq-5.2.0	0.219	0.074	0.187	0.196	0.074	0.149	0.273	0.223	0.108	0.148
activemq-5.3.0	0.202	0.076	0.223	0.140	0.076	0.206	0.290	0.161	0.111	0.207
activemq-5.8.0	0.153	0.032	0.129	0.170	0.028	0.112	0.193	0.182	0.050	0.119
derby-10.2.1.6	0.311	0.310	0.440	0.253	0.286	0.367	0.282	0.212	0.342	0.365
derby-10.3.1.4	0.283	0.296	0.359	0.166	0.267	0.344	0.256	0.170	0.316	0.332
derby-10.5.1.1	0.247	0.125	0.266	0.208	0.111	0.211	0.320	0.228	0.162	0.274
groovy-1_5_7	0.051	0.022	0.129	0.042	0.020	0.050	0.212	0.264	0.026	0.057
groovy-1_6beta1	0.153	0.106	0.214	0.083	0.097	0.172	0.234	0.171	0.133	0.165
groovy-1_6beta2	0.153	0.118	0.238	0.095	0.106	0.146	0.233	0.182	0.143	0.154
hbase-0.94.0	0.303	0.195	0.303	0.117	0.168	0.345	0.254	0.210	0.237	0.346
hbase-0.95.0	0.339	0.297	0.375	0.275	0.288	0.420	0.274	0.241	0.342	0.384
hbase-0.95.2	0.353	0.328	0.412	0.202	0.307	0.458	0.249	0.204	0.367	0.427
hive-0.10.0	0.354	0.112	0.281	0.028	0.097	0.261	0.347	0.308	0.168	0.277
hive-0.12.0	0.201	0.117	0.177	0.000	0.105	0.197	0.147	0.140	0.193	0.199
hive-0.9.0	0.393	0.178	0.326	0.233	0.176	0.249	0.298	0.222	0.226	0.271
jrubby-1.1	0.237	0.085	0.268	0.124	0.074	0.117	0.415	0.298	0.131	0.188
jrubby-1.4.0	0.342	0.177	0.336	0.162	0.158	0.306	0.322	0.153	0.242	0.327
jrubby-1.5.0	0.145	0.057	0.250	0.045	0.047	0.102	0.325	0.234	0.096	0.134
jrubby-1.7.0	0.163	0.040	0.222	0.188	0.030	0.084	0.248	0.190	0.067	0.157
lucene-2.3.0	0.276	0.268	0.318	0.112	0.226	0.341	0.233	0.183	0.301	0.336
lucene-2.9.0	0.279	0.203	0.257	0.256	0.185	0.304	0.257	0.254	0.254	0.304
lucene-3.0.0	0.256	0.132	0.249	0.282	0.105	0.276	0.183	0.189	0.170	0.296
lucene-3.1	0.084	0.036	0.076	0.046	0.029	0.089	0.190	0.190	0.056	0.094
wicket-1.3.0beta2	0.212	0.052	0.165	0.127	0.035	0.141	0.246	0.190	0.067	0.158
wicket-1.3.0beta1	0.166	0.038	0.150	0.139	0.032	0.108	0.239	0.174	0.047	0.120
wicket-1.5.3	0.137	0.022	0.069	0.167	0.022	0.061	0.181	0.133	0.039	0.060
Median	0.228	0.107	0.235	0.154	0.094	0.197	0.263	0.207	0.143	0.212

3.2 Detailed Results for RQ2

Tables 6 and 7 report the median results of the ICUSDP and supervised methods in terms of MCC and F1@20%LOC across all 28 datasets. The NPSKESD test is applied to assess statistical significance and to identify groups of methods that are not significantly different.

In the supervised setting, ICUSDP attains lower median values than the supervised methods on both MCC and F1@20%LOC. This gap is expected, as supervised methods can exploit ground-truth defect labels during training, whereas ICUSDP only learn useful feature representations without such guidance. Nevertheless, ICUSDP achieves statistically competitive results on several individual datasets despite requiring no defect labels. This dataset-level competitiveness indicates that its label-free learning strategy can capture useful defect-related patterns in certain project contexts.

Overall, these results indicate that ICUSDP provides competitive performance relative to supervised methods and remains applicable to defect prediction scenarios where historical defect labels are unavailable or limited. More broadly, the findings suggest a practical trade-off: although fully supervised methods retain an advantage in median performance,

ICUSDP offers a viable alternative when labels are scarce or absent, and the remaining gap highlights opportunities for further improving unsupervised defect prediction.

Table 6 Comparison results of supervised methods using the NPSKESD test in terms of MCC.

Dataset	DT	GBM	linearSVM	LR	RF	XGBoost	ICUSDP
activemq-5.0.0	0.520	0.489	0.477	0.490	0.570	0.510	0.365
activemq-5.1.0	0.194	0.206	0.194	0.276	0.284	0.261	0.247
activemq-5.2.0	0.432	0.362	0.240	0.289	0.431	0.406	0.232
activemq-5.3.0	0.283	0.291	0.294	0.335	0.350	0.317	0.295
activemq-5.8.0	0.252	0.236	0.288	0.315	0.290	0.263	0.242
derby-10.2.1.6	0.398	0.422	0.486	0.489	0.510	0.460	0.360
derby-10.3.1.4	0.338	0.411	0.394	0.412	0.501	0.433	0.338
derby-10.5.1.1	0.298	0.321	0.333	0.372	0.381	0.359	0.344
groovy-1_5_7	0.240	0.199	0.255	0.359	0.374	0.376	0.206
groovy-1_6beta1	0.215	0.271	0.192	0.231	0.306	0.316	0.225
groovy-1_6beta2	0.321	0.342	0.223	0.252	0.451	0.333	0.199
hbase-0.94.0	0.320	0.332	0.321	0.381	0.386	0.382	0.325
hbase-0.95.0	0.326	0.361	0.326	0.348	0.411	0.338	0.230
hbase-0.95.2	0.291	0.305	0.312	0.344	0.384	0.324	0.303
hive-0.10.0	0.276	0.308	0.276	0.289	0.357	0.339	0.303
hive-0.12.0	0.129	0.214	0.091	0.114	0.260	0.172	0.229
hive-0.9.0	0.212	0.310	0.406	0.423	0.545	0.383	0.202
jrubby-1.1	0.391	0.399	0.433	0.465	0.527	0.487	0.379
jrubby-1.4.0	0.380	0.361	0.316	0.419	0.471	0.425	0.415
jrubby-1.5.0	0.295	0.282	0.290	0.324	0.362	0.330	0.310
jrubby-1.7.0	0.191	0.197	0.213	0.295	0.222	0.212	0.273
lucene-2.3.0	0.601	0.625	0.547	0.589	0.703	0.537	0.276
lucene-2.9.0	0.266	0.320	0.208	0.254	0.387	0.316	0.260
lucene-3.0.0	0.282	0.336	0.238	0.254	0.365	0.284	0.293
lucene-3.1	0.163	0.152	0.113	0.191	0.163	0.164	0.150
wicket-1.3.0beta2	0.178	0.216	0.184	0.221	0.254	0.242	0.265
wicket-1.3.0beta1	0.238	0.243	0.220	0.290	0.287	0.308	0.241
wicket-1.5.3	0.087	0.095	0.065	0.156	0.107	0.151	0.163
Median	0.280	0.296	0.278	0.327	0.373	0.332	0.268

Table 7 Comparison results of supervised methods using the NPSKESD test in terms of F1@20%LOC.

Dataset	DT	GBM	linearSVM	LR	RF	XGBoost	ICUSDP
activemq-5.0.0	0.472	0.484	0.438	0.473	0.532	0.495	0.227
activemq-5.1.0	0.221	0.260	0.190	0.167	0.199	0.281	0.145
activemq-5.2.0	0.429	0.406	0.264	0.241	0.403	0.434	0.148
activemq-5.3.0	0.308	0.322	0.279	0.314	0.328	0.341	0.207
activemq-5.8.0	0.188	0.274	0.186	0.140	0.196	0.264	0.119
derby-10.2.1.6	0.451	0.467	0.481	0.491	0.507	0.480	0.365
derby-10.3.1.4	0.391	0.450	0.402	0.410	0.486	0.445	0.332
derby-10.5.1.1	0.306	0.329	0.323	0.348	0.348	0.349	0.274

groovy-1_5_7	0.190	0.191	0.214	0.242	0.171	0.286	0.057
groovy-1_6beta1	0.268	0.313	0.218	0.233	0.274	0.343	0.165
groovy-1_6beta2	0.328	0.385	0.231	0.263	0.312	0.364	0.154
hbase-0.94.0	0.379	0.389	0.367	0.411	0.404	0.431	0.346
hbase-0.95.0	0.433	0.485	0.404	0.399	0.439	0.448	0.384
hbase-0.95.2	0.430	0.431	0.407	0.418	0.462	0.427	0.427
hive-0.10.0	0.316	0.354	0.308	0.269	0.326	0.366	0.277
hive-0.12.0	0.234	0.283	0.126	0.183	0.220	0.256	0.199
hive-0.9.0	0.299	0.364	0.381	0.403	0.510	0.399	0.271
jruby-1.1	0.351	0.353	0.369	0.390	0.449	0.426	0.188
jruby-1.4.0	0.361	0.357	0.314	0.357	0.396	0.374	0.327
jruby-1.5.0	0.295	0.297	0.270	0.300	0.319	0.322	0.134
jruby-1.7.0	0.183	0.204	0.204	0.261	0.148	0.236	0.157
lucene-2.3.0	0.638	0.657	0.586	0.619	0.721	0.584	0.336
lucene-2.9.0	0.345	0.396	0.290	0.293	0.406	0.382	0.304
lucene-3.0.0	0.360	0.400	0.331	0.335	0.335	0.405	0.296
lucene-3.1	0.099	0.156	0.063	0.059	0.112	0.159	0.094
wicket-1.3.0beta2	0.205	0.257	0.183	0.144	0.209	0.274	0.158
wicket-1.3.0beta1	0.236	0.274	0.184	0.189	0.232	0.333	0.120
wicket-1.5.3	0.093	0.136	0.084	0.058	0.091	0.165	0.060
Median	0.318	0.344	0.293	0.312	0.350	0.364	0.212

3.3 Results of the ICUSDP Feature Category Distribution

Figure 1 illustrates the category distribution of the top-3@SKESD software metrics identified by ICUSDP. Code metrics constitute the majority of the selected metrics (80%), followed by ownership metrics (15%) and process metrics (5%). Table 8 further lists the top-3@SKESD metrics along with their rankings, categories, and semantic descriptions. Consistent with the top-3@SKESD distribution, most of the top-3@SKESD metrics are code metrics, whereas ownership and process metrics account for a smaller proportion. This consistency suggests that the dominance of code metrics is not merely an artifact of the top-3@SKESD cutoff, but remains stable when a broader set of highly ranked metrics is considered. It also indicates that ICUSDP primarily relies on code-level structural properties for defect discrimination, while ownership and process metrics serve as complementary but less dominant signals. Collectively, these results provide additional insight into the types and semantic characteristics of the software metrics identified by ICUSDP, and further support the plausibility of its learned defect-discriminative representations.

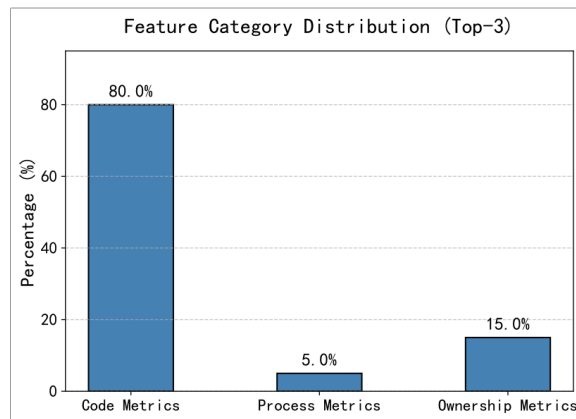


Figure 1 Distribution of metric category

Table 8 Top-3@SKESD Software Metrics Identified by ICUSDP

Rank	Metric	Category	Semantic
1	PercentLackOfCohesion	Code metrics	the 100% minus the average cohesion for package entities
2	CountClassCoupled	Code metrics	the number of other classes to which a class is coupled
3	MaxNesting_Mean	Code metrics	the maximum nesting level of control constructs
4	RatioCommentToCode	Code metrics	the ratio of comment lines to code lines
5	CountLineComment	Code metrics	the number of lines containing comment
6	AvgCyclomaticModified	Code metrics	the modified McCabe Cyclomatic complexity
7	CountOutput_Mean	Code metrics	the number of called subprograms plus global variables set
8	CountDeclClassVariable_Mean	Code metrics	the number of class methods
9	CountDeclInstanceVariable_Mean	Code metrics	the number of instance variables
10	OWN_COMMIT	Ownership metrics	the proportion of code changes (i.e., Git commits) made by the developer who has the highest contribution of code changes on the module
11	CountDeclMethodPublic	Code metrics	the number of local (not inherited) public methods
12	MAJOR_COMMIT	Ownership metrics	the number of unique developers who have contributed more than 5% of the total code changes (i.e., Git commits) on the module
13	CountDeclMethodPrivate	Code metrics	the number of local (not inherited) private methods
14	CountDeclInstanceMethod	Code metrics	the number of instance methods
15	CountClassBase	Code metrics	the number of immediate base classes
16	MAJOR_LINE	Ownership metrics	the number of unique developers who have contributed more than 5% of the total lines of code on the module
17	Added_Lines	Process metrics	the normalized number of lines added to the module
18	AvgEssential	Code metrics	the average essential complexity for all nested functions or methods
19	AvgLineBlank	Code metrics	the average number of blank lines for all nested functions or methods
20	CountDeclClassMethod	Code metrics	the number of class methods

3.4 Results of the Top-k@SKESD overlap and Kendall's Tau.

Table 9 further compares the feature rankings generated by ICUSDP with those of 6 supervised models. The top-1@SKESD overlap values range from 0.030 to 0.250, while the top-3@SKESD overlap values range from 0.106 to 0.224, indicating low or negligible direct overlap between the rankings. In contrast, ICUSDP exhibits moderate positive Kendall's Tau correlations with DT (0.396), RF (0.566), GBM (0.491), and XGBoost (0.403), all with statistically significant p-values. The correlations with LR and linearSVM are weak and not statistically significant. These results suggest that while ICUSDP does not consistently identify exactly the same top-ranked metrics as supervised models, its overall feature-ranking patterns show partial consistency, particularly with tree-based models.

Table 9 Comparison of feature-ranking agreement between ICUSDP and supervised methods in top-k@SKESD overlap and Kendall's Tau.

Method	Top-1@SKESD Overlap	Top-1@SKESD Agreement	Top-3@SKESD Overlap	Top-3@SKESD Agreement	Kendall's Tau	p-value	Tau Agreement
DT	0.196	low	0.224	negligible	0.396	0.001	moderate
RF	0.250	low	0.212	negligible	0.566	1.14E-06	moderate
GBM	0.196	low	0.218	negligible	0.491	2.21E-05	moderate
XGBoost	0.125	low	0.106	negligible	0.403	4.75E-04	moderate
LR	0.074	low	0.173	negligible	0.085	0.464	weak
linearSVM	0.030	low	0.142	negligible	0.163	0.161	weak

3.5 Results of the Generalizability Evaluation for the ICUSDP Framework

Table 10 presents a generalizability evaluation of the ICUSDP framework by replacing its feature learner, clustering model, and interpreter with alternative components. The results show that different feature learners, clustering models, and interpreters can be incorporated into the framework while still producing valid prediction results. The evaluated alternatives exhibit varying performance across G-mean, MCC, F1@20%LOC, and IFA, indicating that the choice of individual components can affect the resulting performance. This variation is expected, since each component introduces different inductive biases, optimization behavior, and interpretation mechanisms. Nevertheless, the framework remains applicable under these different component configurations, suggesting that its core explanation-driven joint optimization strategy is not tied to a single algorithmic instantiation.

These results provide empirical evidence that ICUSDP is not tied to a specific implementation of its individual components and can be extended to alternative learning, clustering, and interpretation modules. More broadly, this component-level robustness supports viewing ICUSDP as a flexible framework rather than a fixed pipeline. It also implies that the framework’s effectiveness stems primarily from the interaction between prediction and explanation-driven optimization, rather than from any single component alone. At the same time, the observed performance differences highlight that component selection is not neutral and may need to be tailored to specific data characteristics, project contexts, or evaluation goals.

Practically, this flexibility allows researchers and practitioners to instantiate ICUSDP with components that best match their computational budget, interpretability requirements, or prediction objectives. It also suggests several directions for future work, such as systematically benchmarking a broader range of components, developing adaptive component-selection strategies, and designing new interpreters specifically for defect prediction. Overall, these findings strengthen the claim that ICUSDP is a generalizable and extensible framework for unsupervised defect prediction and interpretation.

Table 10 Results of the Generalizability Evaluation for the ICUSDP Framework

	Method	G_mean	MCC	F1@20%LOC	IFA
	ICUSDP	0.699	0.268	0.212	10.000
Feature learner	Modify into AE	0.445	0.179	0.194	8.000
	Modify into DAE	0.480	0.195	0.202	9.000
	Modify into CAE	0.701	0.272	0.202	10.000
	Modify into SC	0.680	0.237	0.176	15.000
Clustering	Modify into Agglomerative	0.595	0.200	0.184	13.000
	Modify into GMM	0.655	0.231	0.202	14.000
	Modify into K-medoids	0.557	0.077	0.130	33.000
	Modify into RF	0.699	0.268	0.211	10.000
Interpreter	Modify into GBM	0.699	0.268	0.212	10.000
	Modify into XGBoost	0.699	0.268	0.211	10.000

Reference

- [1] S. Yatish, J. Jiarpakdee, P. Thongtanunam, and C. Tantithamthavorn, “Mining software defects: Should we consider affected releases?” in ICSE’19, 2019, pp. 654–665.
- [2] J. Jiarpakdee, C. Tantithamthavorn, and A. Hassan, “The impact of correlated metrics on the interpretation of defect models,” IEEE Transactions on Software Engineering, vol. 47, no. 2, pp. 320–331, 2021.
- [3] J. Jiarpakdee, C. Tantithamthavorn, H. K. Dam, and J. Grundy, “An empirical study of model-agnostic techniques for defect prediction models,” IEEE Transactions on Software Engineering, vol. 48, no. 1, pp. 166–185, 2022.

- [4] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "The impact of automated parameter optimization on defect prediction models," *IEEE Transactions on Software Engineering*, vol. 45, no. 7, pp. 683–711, 2019.
- [5] C. Tantithamthavorn, A. E. Hassan, and K. Matsumoto, "The impact of class rebalancing techniques on the performance and interpretation of defect prediction models," *IEEE Transactions on Software Engineering*, vol. 46, no. 11, pp. 1200–1219, 2020.
- [6] J. Jiarapakdee, C. Tantithamthavorn, and C. Treude, "Autospearman: Automatically mitigating correlated software metrics for interpreting defect models," in *ICSME'18. IEEE*, 2018, pp. 92–103.